

Module 5

Memory Management

Contiguous Memory Location

Introduction

- The computer's memory must hold both the operating system (which controls the computer) and the various programs (or processes) that users are running.
- The memory is typically divided into **two main sections**: one for the operating system and one for user processes.
- Often, several user processes are running at the same time. This means the system needs to decide how to allocate the available memory to these processes.

Contiguous Memory Location

In contiguous memory allocation, each process is assigned a **single, continuous** section of memory and these sections are placed next to each other.

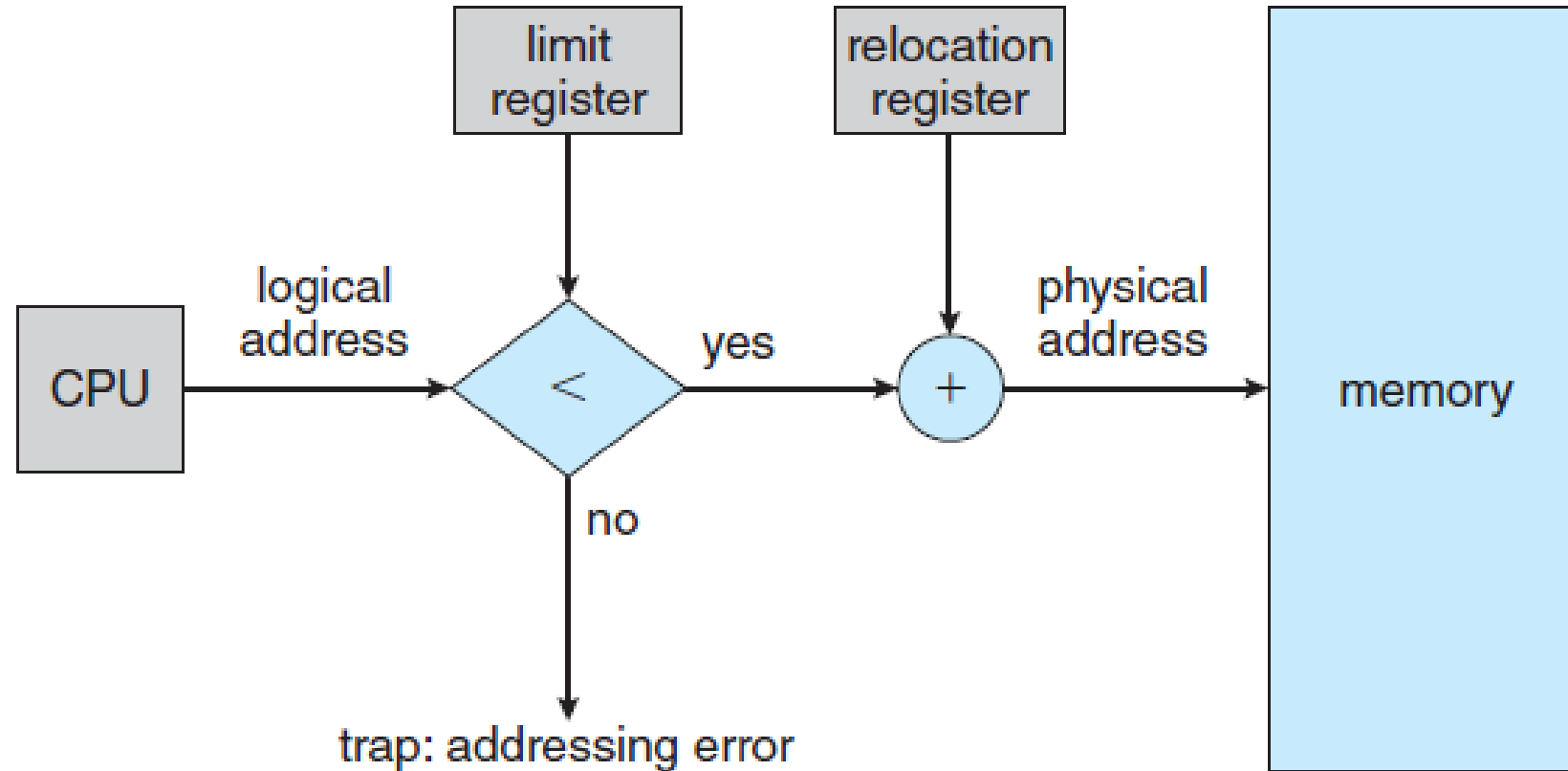
Memory Protection

When the CPU scheduler selects a process for execution, the **dispatcher** loads the relocation and limit registers.

Every address generated by a CPU is checked against these registers

The relocation-register scheme provides an effective way to allow the operating system's size to change dynamically.

Memory Protection



Memory Location - Fixed/Static Partitioning

- In this method, the memory is divided into fixed-size blocks before any processes are loaded.
- A problem arises when a process doesn't perfectly fit into a block. This leaves unused gaps called "fragments," which can waste memory.
- Fragmentation makes it harder to allocate memory to new processes, and it can reduce the overall available memory.

Memory Location - **Dynamic Partitioning**

- One of the simplest methods of allocating memory is to assign processes to **variably sized partitions in memory**, where each partition may contain exactly one process.
- The operating system keeps track of which parts of memory are available and which are occupied using a memory table.

Memory Location - Dynamic Partitioning

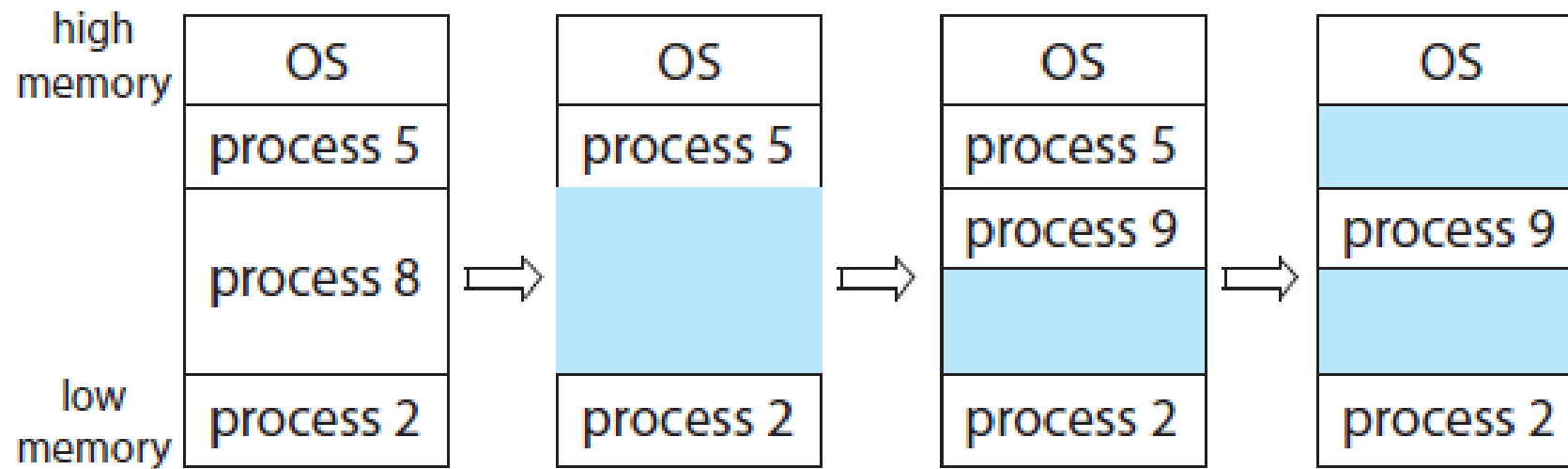


Figure: Variable partition.

Memory Location - **Dynamic Partitioning**

- Initially, all memory is considered one large "hole" that can be divided into smaller holes as needed.
- When a process arrives, the system searches for a hole that's big enough to fit it. If the hole is too large, it's split into two parts.
- When a process finishes, its memory is returned to the set of holes, which can then be merged with adjacent holes to create larger ones.

Memory Location - **Dynamic Partitioning**

- The dynamic storage allocation problem involves effectively managing memory by satisfying requests for memory blocks of varying sizes from a list of available memory blocks (holes).
- A hole is a block of contiguous memory that is currently unoccupied. It's available for allocation to processes or other data structures.

Dynamic Storage Allocation Problem

How to satisfy a request of size **n** from a list of free holes?

Three strategies

first-fit, best-fit, and worst-fit strategies are the ones most commonly used to select a free hole from the set of available holes.

Dynamic Storage Allocation Problem

First fit. Allocate the first hole that is big enough. Searching can start either at the beginning of the set of holes or at the location where the previous first-fit search ended.

For example, suppose we have the following memory partitions:

| 10 KB | 20 KB | 19 KB | 25 KB | 30 KB |

Now, a process requests **18 KB of memory**. The OS starts searching from the beginning and finds the first free partition of 20 KB. It allocates the process to that partition and keeps the remaining 2 KB as free memory.

Dynamic Storage Allocation Problem

Best fit. Allocate the smallest hole that is big enough. We must search the entire list, unless the list is ordered by size.

For example, suppose we have the following memory partitions:

| 10 KB | 20 KB | 19 KB | 25 KB | 30 KB |

Now, a process requests **18 KB of memory**. The OS starts searching from the beginning and finds the partition of 19 KB.

Dynamic Storage Allocation Problem

Worst fit. Allocate the largest hole. Again, we must search the entire list, unless it is sorted by size.

For example, suppose we have the following memory partitions:

| 10 KB | 20 KB | 19 KB | 25 KB | 30 KB |

Now, a process requests 18 KB of memory. The operating system searches for the largest free partition, which is 30 KB. It allocates the process to that partition and keeps the remaining 12 KB as free memory.

Fragmentation

- First Fit is fast and simple to implement
- first-fit and best-fit strategies for memory allocation suffer from **External fragmentation**
- External fragmentation exists when there is enough total memory space to satisfy a request but the **available spaces are not contiguous**

Fragmentation

- Statistical analysis of first fit, reveals that, even with some optimization, given N allocated blocks, another **$0.5 N$ blocks will be lost** to fragmentation.
- That is, one-third of memory may be unusable. This property is known as the **50-percent rule**.
- This will increase the overhead so as to keep track of this holes
- To avoid, break the physical memory into fixed-sized blocks and allocate memory in units based on block size

Fragmentation

- With this approach, the memory allocated to a process may be slightly larger than the requested memory. The difference between these two numbers is **internal fragmentation** — unused memory that is internal to a partition.
- One solution to the problem of external fragmentation is **compaction**. The goal is to shuffle the memory contents so as to place all free memory together in one large block

Fragmentation

- Another possible solution to the external-fragmentation problem is to **permit the logical address space of processes to be noncontiguous**, thus allowing a process to be allocated physical memory wherever such memory is available.
- This is the strategy used in **paging**, the most common memory-management technique for computer systems.

Contiguous Memory Allocation Schemes – Problem 1

Suppose you have a memory with the following free blocks: 12KB, 6KB, 14KB, 19KB, 21KB. Four processes arrive that require 10KB, 4KB, 7KB, and 18KB of memory, respectively. Using the different allocation techniques, how the process would be allocated to memory.

Contiguous Memory Allocation Schemes – Problem 1 – **First Fit**

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Block Status	Memory Wastage
		B1	12KB	Yes			
		B2	6KB	Yes			
		B3	14KB	Yes			
		B4	19KB	Yes			
		B5	21KB	Yes			

Contiguous Memory Allocation Schemes – Problem 1 - **First Fit**

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Block Status	Memory Wastage
P1	10KB	B1	12KB	Yes	Check $10 < 12$?	Allocated	$12 - 10 = 2$
		B2	6KB	Yes			
		B3	14KB	Yes			
		B4	19KB	Yes			
		B5	21KB	Yes			

Contiguous Memory Allocation Schemes – Problem 1 - **First Fit**

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Block Status	Memory wastage
P1	10KB	B1	12KB	Yes	Check $10 \leq 12$?	Allocated	$12 - 10 = 2\text{KB}$
P2	10KB	B1	12KB	No	No		
P2	4KB	B2	6KB	Yes	Check $4 \leq 6$	Allocated	$6 - 4 = 2\text{KB}$
P3	7KB	B1	12KB	No	No		
		B2	6KB	No	No		
		B3	14KB	Yes	Check $7 \leq 14$	Allocated	$17 - 4 = 7\text{KB}$
P4	18KB	B1	12KB	No			
		B2	6KB	No			
		B3	14KB	No			
		B4	19KB	Yes	Check $18 \leq 19$	Allocated	$19 - 18 = 1\text{KB}$
		B5	21KB	Yes			

Contiguous Memory Allocation Schemes – Problem 1 - Next Fit

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Block Status	Memory Balance
P1	10KB	Block 1	12KB	Available	Check $10 < 12$	P1 Allotted	$12 - 10 = 2\text{KB}$
P2	4KB	Block 2	6KB	Available	Check $4 < 6$	P2 Allotted	$6 - 4 = 2\text{KB}$
P3	7KB	Block 3	14KB	Available	Check $7 < 14$	P3 Allotted	$14 - 7 = 7\text{KB}$
P4	18KB	Block 4	19 KB	Available	Check $18 < 19$	P4 Allotted	$19 - 18 = 1\text{KB}$
		Block 5	21KB	Available		Empty	

Contiguous Memory Allocation Schemes – Problem 1 - **Best Fit**

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Memory Wastage	Find Minimum Wastage Block	Block Status
P1	10KB	Block 1	12KB	Available	Check $10 < 12$	$12 - 10 = 2\text{KB}$	$2 < 4 < 9 < 11$ Block 1	P1 Allotted
		Block 2	6KB	Available	Check $10 < 6$	Not Fitting		Empty
		Block 3	14KB	Available	Check $10 < 14$	$14 - 10 = 4\text{KB}$		Empty
		Block 4	19 KB	Available	Check $10 < 19$	$19 - 10 = 9\text{KB}$		Empty
		Block 5	21KB	Available	Check $21 < 10$	$21 - 10 = 11\text{KB}$		Empty

Contiguous Memory Allocation Schemes – Problem 1 - Best Fit

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Memory Wastage	Find Minimum Wastage Block	Block Status
P2	4KB	Block 1	12KB	Unavailable			$2 < 10 < 15 < 17$ Block 2	P1 Allotted
		Block 2	6KB	Available	Check $4 < 6$	$6 - 4 = 2\text{KB}$		P2 Allotted
		Block 3	14KB	Available	Check $4 < 14$	$14 - 4 = 10\text{KB}$		Empty
		Block 4	19 KB	Available	Check $4 < 19$	$19 - 4 = 15\text{KB}$		Empty
		Block 5	21KB	Available	Check $4 < 21$	$21 - 4 = 17\text{KB}$		Empty

Contiguous Memory Allocation Schemes – Problem 1 - **Best Fit**

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Memory Wastage	Find Minimum Wastage Block	Block Status
P3	7KB	Block 1	12KB	Unavailable			$7 < 12 < 14$ Block 3	P1 Allotted
		Block 2	6KB	Unavailable				P2 Allotted
		Block 3	14KB	Available	Check $7 < 14$	$14 - 7 = 7\text{KB}$		P3 Allotted
		Block 4	19 KB	Available	Check $7 < 19$	$19 - 7 = 12\text{KB}$		Empty
		Block 5	21KB	Available	Check $7 < 21$	$21 - 7 = 14\text{KB}$		Empty

Contiguous Memory Allocation Schemes – Problem 1 - **Best Fit**

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Memory Wastage	Find Minimum Wastage Block	Block Status
P4	18KB	Block 1	12KB	Unavailable			1 < 3 Block 4	P1 Allotted
		Block 2	6KB	Unavailable				P2 Allotted
		Block 3	14KB	Unavailable				P3 Allotted
		Block 4	19 KB	Available	Check $18 < 19$	$19 - 18 = 1\text{KB}$		P4 Allotted
		Block 5	21KB	Available	Check $7 < 21$	$21 - 18 = 3\text{KB}$		Empty

Contiguous Memory Allocation Schemes – Problem 1 - **Worst Fit**

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Memory Wastage	Find Minimum Wastage Block	Block Status
P1	10KB	Block 1	12KB	Available	Check $10 < 12$	$12 - 10 = 2\text{KB}$	$11 > 9 > 4 > 2$ Block 5	Empty
		Block 2	6KB	Available	Check $10 < 6$	Not Fitting		Empty
		Block 3	14KB	Available	Check $10 < 14$	$14 - 10 = 4\text{KB}$		Empty
		Block 4	19 KB	Available	Check $10 < 19$	$19 - 10 = 9\text{KB}$		Empty
		Block 5	21KB	Available	Check $21 < 10$	$21 - 10 = 11\text{KB}$		P1 Allotted

Contiguous Memory Allocation Schemes – Problem 1 - Worst Fit

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Memory Wastage	Find Minimum Wastage Block	Block Status
P2	4KB	Block 1	12KB	Available	Check $4 < 12$	$12 - 4 = 8\text{KB}$	$15 > 10 > 8 > 7 > 2$ Block 4	Empty
		Block 2	6KB	Available	Check $4 < 6$	$6 - 4 = 2\text{KB}$		Empty
		Block 3	14KB	Available	Check $4 < 14$	$14 - 4 = 10\text{KB}$		Empty
		Block 4	19 KB	Available	Check $4 < 19$	$19 - 4 = 15\text{KB}$		P2 Allotted
		Block 5	21KB	Not available				P1 Allotted
		Block 5a	11KB	Available	Check $4 < 14$	$11 - 4 = 7\text{KB}$		Empty

Contiguous Memory Allocation Schemes – Problem 1 - Worst Fit

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Memory Wastage	Find Minimum Wastage Block	Block Status
P3	7KB	Block 1	12KB	Available	Check $7 < 12$	$12 - 7 = 5\text{KB}$	$7 > 5$ Block 3	
		Block 2	6KB	Available	Check $7 < 6$	Not Fitting		
		Block 3	14KB	Available	Check $7 < 14$	$14 - 7 = 7\text{KB}$		P3 Allotted
		Block 4	19 KB	Unavailable				P2 Allotted
		Block 4a		Available				P1 Allotted
		Block 5	21KB	Unavailable				
		Block 5a		Available				

Contiguous Memory Allocation Schemes – Problem 1 - **Worst Fit**

Process No.	Process Size	Block No.	Block Size	Availability Status	Can Occupy Process ?	Memory Wastage	Find Minimum Wastage Block	Block Status
P4	18KB	Block 1	12KB	Available	Check $18 < 12$	Not Fitting	No Large Block Available, Reject the process P4 or make it to Wait P4 remains Unallocated	Empty
		Block 2	6KB	Available	Check $18 < 6$	Not Fitting		Empty
		Block 3	14KB	Unavailable				P3 Allotted
		Block 4	19 KB	Unavailable				P2 Allotted
		Block 5	21KB	Unavailable				P1 Allotted

Problem for Practice

Suppose you have a memory with the following free blocks: 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB. Four processes arrive that require 357 KB, 210 KB, 468 KB and 491 KB of memory, respectively. Using the different allocation techniques, how the process would be allocated to memory.